



UNIVERSITÀ DEGLI STUDI DI FIRENZE
INGEGNERIA INFORMATICA

Implementazione di Statistiche d'Ordine Dinamiche

Autore:
Neri Saldutti

Corso:
Algoritmi e Strutture Dati

N° Matricola:
7171388

Docente:
Simone Marinai

a.a. 2025/2026

Indice

1	Introduzione	1
1.1	Obiettivo	1
1.2	Specifiche delle condizioni sperimentali	1
2	Fondamenti teorici	2
2.1	Statistiche d'ordine dinamiche	2
2.2	Prestazioni attese	2
3	Documentazione	4
3.1	Architettura del codice	4
3.2	Implementazione delle strutture dati	4
3.3	Implementazione degli esperimenti	8
4	Esperimenti e risultati	8
4.1	Presentazione dei risultati	8
4.2	Analisi e discussione	8
5	Conclusioni	8

Elenco delle figure

1	Diagramma UML delle tre implementazioni	7
---	---	---

Elenco delle tabelle

1	Complessità lista ordinata	2
2	Complessità albero binario di ricerca	3
3	Complessità albero binario di ricerca	3

1 Introduzione

1.1 Obiettivo

La presente relazione ha l'obiettivo di descrivere l'implementazione e l'analisi sperimentale delle *statistiche d'ordine dinamiche*, realizzate in linguaggio *Python* estendendo tre strutture dati distinte: una *lista ordinata*, un *albero binario di ricerca* (ABR) e un *albero AVL*.

L'obiettivo è confrontare le prestazioni delle tre implementazioni sottoponendole a test differenti, analizzando l'andamento ottenuto dei tempi di esecuzione e verificandone la coerenza con la complessità asintotica attesa dall'analisi teorica.

1.2 Specifiche delle condizioni sperimentali

Si riportano le specifiche hardware del calcolatore su cui sono stati svolti i test:

- **CPU:** AMD Ryzen 5 5600H 6 core 12 threads
- **GPU:** AMD Radeon Graphics Vega 7 e NVIDIA GeForce RTX 3050
- **SSD:** SSD M.2 PCIe NVMe 512 GB
- **RAM:** 16 GB DDR4-3200

Il codice è stato eseguito utilizzando **Python 3.14.4** e l'IDE **PyCharm 2026.1** su sistema operativo **Arch Linux**. La relazione è stata redatta utilizzando $\text{\TeX}$ e $\text{\LaTeX}$ l'editor offline **TeXStudio v4.9.3**. Gli schemi UML sono stati realizzati utilizzando **mermaid.js**.

2 Fondamenti teorici

2.1 Statistiche d'ordine dinamiche

Le statistiche d'ordine dinamiche sono delle particolari strutture dati aumentate che permettono di aggiungere due funzionalità principali:

- $OS-SELECT(i)$ che ritorna il puntatore al nodo con l' i -esima chiave più piccola
- $OS-RANK(x)$ che ritorna il rango di un determinato nodo a partire da un attraversamento ordinato

L'idea di base delle strutture dati aumentate consiste nel partire da una struttura esistente ed estenderla, ad esempio aggiungendo nuovi attributi ad ognuno dei suoi nodi, in modo da poter implementare in maniera efficiente nuovi metodi.

Nel caso specifico di questa relazione, ci interessa che sia possibile eseguire le operazioni di *inserimento*, *eliminazione*, *ranking* e *selezione*.

2.2 Prestazioni attese

Le prestazioni delle operazioni eseguite dipendono dalla struttura dati sottostante e della scelte implementative adottate. Si prosegue andando ad analizzare ognuna delle implementazioni, analizzando nello specifico il caso peggiore, il caso medio e il caso migliore.

Lista Ordinata

Una lista ordinata è una struttura dati che mantiene i suoi elementi in ordine, in questo caso crescente. Il costo delle singole operazioni dipende dalla posizione dell'elemento su cui si vuole operare. Considerata una lista di n elementi, si ha che nel caso migliore, l'operazione coinvolge solamente il primo elemento e si avrà quindi una complessità di $\Theta(1)$ mentre nel caso peggiore dovrà scorrere fino all'ultimo elemento e la complessità sarà dunque $\Theta(n)$. Per il caso medio si dovrà invece fare un'analisi probabilistica. Assumendo un input distribuito in maniera uniforme, il caso medio ci richiederà di scorrere circa $\frac{n}{2}$ elementi, risultando in una complessità di $O(n)$.

Per le specifiche operazioni si rimanda alla Tabella 1.

	Caso peggiore	Caso medio	Caso migliore
Inserimento	$O(n)$	$O(n)$	$O(1)$
Eliminazione	$O(n)$	$O(n)$	$O(1)$
Ricerca	$\Theta(1)$	$O(n)$	$\Theta(n)$
Ranking	$O(n)$	$O(n)$	$O(1)$
Selezione	$\Theta(n)$	$O(n)$	$\Theta(1)$

Tabella 1: Complessità lista ordinata

Albero Binario di Ricerca

Un albero binario di ricerca, comunemente abbreviato con ABR o dall'inglese BST, è una struttura costituita da nodi disposti in modo gerarchico. Ogni albero ha un nodo che svolge il ruolo di *radice* (*root*) e costituisce l'inizio della struttura. Poi ogni nodo presenta tre puntatori: uno al figlio destro,

uno al figlio sinistro e uno al genitore. Ciò che caratterizza un ABR da altri tipi di alberi, è che l'unica regola di ordinamento è che la chiave (o il valore) del figlio sinistro di un nodo deve essere minore della chiave del nodo, mentre la chiave del figlio destro di nodo deve essere maggiore della chiave del nodo.

La complessità di un ABR dipende dunque dalla sua struttura interna e in particolare dalla sua altezza h , influenzata a sua volta dall'ordine di inserimento degli elementi.

Nel caso peggiore si avrà che il sottoalbero sinistro della radice contiene tutti gli n elementi dell'albero, mentre il sottoalbero destro resta vuoto. In questo caso $h = n$. Nel caso migliore invece si avrà un albero perfettamente bilanciato in cui si può dimostrare che l'altezza è $h = \ln(n)$.

Per le specifiche operazioni si rimanda alla Tabella 2.

	Caso peggiore	Caso medio	Caso migliore
Inserimento	$O(n)$	$O(\ln(n))$	$O(1)$
Eliminazione	$\Theta(1)$	$O(n)$	$\Theta(n)$
Ricerca	$\Theta(1)$	$O(n)$	$\Theta(n)$
Ranking	$\Theta(1)$	$O(n)$	$\Theta(n)$
Selezione	$\Theta(1)$	$O(n)$	$\Theta(n)$

Tabella 2: Complessità albero binario di ricerca

Albero AVL

Un albero AVL è invece un albero quasi bilanciato, ovvero che per ogni nodo le altezze dei due sottoalberi destro e sinistro possono differire al massimo di un elemento. In questo tipo di albero si ha che l'altezza di un qualsiasi nodo è dato dal numero di archi nel cammino più lungo fino ad una estremità. Essendo un albero sempre bilanciato si può dimostrare che l'altezza di un AVL che contiene n nodi sarà $h \leq 2 \lg(n + 1)$. Tutte le operazioni che non modificano la struttura dell'albero hanno questa complessità.

Avendo questo vincolo sulle altezze, vengono aggiunti dei metodi chiamati di *ristrutturazione* in particolare il LEFT-ROTATE per la rotazione a sinistra e il RIGHT-ROTATE per la rotazione a destra. Lo scopo di questi metodi è mantenere il *Fattore di Bilanciamento*, o *Balance Factor* ≤ 1 .

Per le specifiche operazioni si rimanda alla Tabella 3.

	Caso migliore	Caso medio	Caso peggiore
Inserimento	$O(n)$	$O(h)$	$\Theta(n)$
Eliminazione	$O(n)$	$O(h)$	$\Theta(n)$
Ricerca	$O(n)$	$O(h)$	$\Theta(n)$
Ranking	$\Theta(1)$	$O(n)$	$\Theta(n)$
Selezione	$\Theta(1)$	$O(n)$	$\Theta(n)$

Tabella 3: Complessità albero binario di ricerca

3 Documentazione

3.1 Architettura del codice

Il codice di laboratorio è diviso in tre parti principali:

- Implementazione delle strutture
- Implementazione dei test da effettuare
- Implementazione del plotter dei grafici

La parte di implementazione delle strutture include tre file principali, ognuno dei quali contiene due classi una per il nodo costituente e una per la struttura in sé. Il file `os_orderd_list.py` che comprende l'implementazione della statistica d'ordine dinamica con lista ordinata, `os_bst.py` quella con l'albero binario di ricerca e `os_avl.py` l'implementazione con l'albero AVL.

Il file `test_runner.py` contiene l'implementazione di tutti quanti i test che devono essere effettuati. È qui che si trova la funzione `main()`.

Per finire troviamo il file `plot_results.py` che si occupa del salvataggio e lettura dei dati grezzi in formato `.csv` oltre che al salvataggio di ogni singolo grafico in formato `.pdf`.

3.2 Implementazione delle strutture dati

Per il diagramma completo delle classi fare riferimento alla Figura 1.

Lista Ordinata

Per l'implementazione tramite lista ordinata si crea una lista doppiamente collegata. Di fatto ogni `OrderedListNode` avrà tre attributi fondamentali: la chiave `key`, il puntatore al nodo successivo `next` e il puntatore al nodo precedente `prev`. Il nodo in testa alla lista avrà `prev = null`, mentre l'ultimo nodo della coda avrà `next = null`.

Di sotto si riporta la spiegazione dei metodi della lista ordinata.

- `insert(key)`: inserisce in maniera ordinata un nuovo elemento di chiave `key` nella lista.
- `delete(key)`: rimuove il primo elemento di chiave `key` dalla lista. Se l'elemento non è presente scorre tutta la lista. In seguito all'eliminazioni vengono aggiustati i puntatori dei nodi adiacenti a quello eliminato in maniera corretta.
- `print(key)`: scorre la lista e stampa ogni elemento in ordine.
- `select(i)`: restituisce la chiave dell'*i*-esimo nodo all'interno della lista
- `rank(key)`: restituisce la posizione a partire dalla testa della lista (rango) del nodo di chiave `key`.

Albero Binario di Ricerca

Il singolo nodo dell'ABR, `BSTNode` è stato implementato con gli attributi: la chiave `key`, il puntatore al figlio destro `right`, il puntatore al figlio sinistro `left` e il puntatore al padre `parent`. Il nodo radice `root` è l'unico che ha il puntatore `parent = null`.

Per l'albero binario di ricerca sono stati implementati i seguenti metodi:

- `insert(key)`: inserisce l'elemento di chiave `key` all'interno dell'albero nella posizione corretta.
- `delete(key)`: rimuove l'elemento di chiave `key` dall'albero effettuando in seguito una ristrutturazione per ricollegare i nodi in maniera corretta. Fa uso del metodo `successor(node)`.
- `print(key)`: metodo implementato per avere la stessa interfaccia fra le tre implementazioni. L'unica cosa che fa è chiamare il metodo `inorder_tree_walk()`.
- `select(i)`: esplora l'albero e restituisce il nodo in posizione `i`-esima rispetto ad un `inorder_tree_walk()`.
- `rank(key)`: esplora l'albero e restituisce la posizione rispetto ad un `inorder_tree_walk()` del nodo di chiave `key`.
- `min(node)`: restituisce il nodo con la chiave più piccola. Per costruzione è il figlio più a sinistra.
- `max(node)`: restituisce il nodo con la chiave più grande. Per costruzione è il figlio più a destra.
- `find(node, key)`: cerca il nodo con la chiave `key`. Se non viene trovato restituisce `None`.
- `successor(node)`: restituisce il successore del nodo `node`. Fa uso del metodo `min(node)`.
- `inorder_tree_walk(node)`: effettua un attraversamento ordinato dell'albero. Stampando gli elementi dal più piccolo al più grande.
- `size(node)`: Conta quanti nodi si trovano nei due sottoalberi destro e sinistro di `node`, compreso `node` stesso.

Albero AVL

Il singolo nodo dell'Albero AVL, `AVLNode` è stato implementato con 5 attributi: la chiave `key`, il puntatore al figlio destro `right`, il puntatore al figlio sinistro `left`, l'attributo `height` che tiene traccia del numero di archi nel percorso più lungo dal nodo alla foglia (primo nodo con entrambi i puntatori ai figli `None`) e l'attributo `size`, che tiene traccia del numero totale di nodi nei sottoalberi destro e sinistro compreso il nodo radice. È l'analogo di `size(node)` dell'ABR, ma in questo caso invece di calcolarlo tutte le volte che mi serve, lo salvo come attributo e lo aggiorni solo quando modifico la struttura dell'albero.

Di sotto in dettaglio i metodi della classe:

- `insert(key)`: inserisce l'elemento di chiave `key` all'interno dell'albero nella posizione corretta. Effettua una rotazione appropriata per mantenere il `balance_factor ≤ 1`, utilizzando i metodi `right_rotate(node)` e `left_rotate(node)`.

- `delete(key)`: rimuove l'elemento di chiave *key* dall'albero effettuando in seguito una ristrutturazione tramite rotazioni per bilanciare i nodi in maniera corretta. Fa uso del metodo `successor(node)`.
- `print(key)`: metodo implementato per avere la stessa interfaccia fra le tre implementazioni. L'unica cosa che fa è chiamare il metodo `inorder_tree_walk()`.
- `select(i)`: esplora l'albero e restituisce il nodo in posizione *i*-esima rispetto ad un `inorder_tree_walk()`.
- `rank(key)`: esplora l'albero e restituisce la posizione rispetto ad un `inorder_tree_walk()` del nodo di chiave *key*.
- `get_height(node)`: restituisce l'altezza del nodo *node*.
- `get_size(node)`: restituisce la dimensione del nodo *node*.
- `get_balance(node)`: restituisce il fattore di bilanciamento, `balance_factor` del nodo *node*.
- `update(node)`: metodo di supporto. Ricalcola e aggiorna gli attributi `height` e `size` del nodo *node*.
- `right_rotate(node)`: effettua una rotazione a destra dell'albero.
- `left_rotate(nide)`: effettua una rotazione a sinistra dell'albero.
- `min(node)`: restituisce il nodo con la chiave più piccola. Per costruzione è il figlio più a sinistra.
- `max(node)`: restituisce il nodo con la chiave più grande. Per costruzione è il figlio più a destra.
- `successor(node)`: restituisce il successore del nodo *node*. Fa uso del metodo `min(node)`.
- `inorder_tree_walk(node)`: effettua un attraversamento ordinato dell'albero. Stampando gli elementi dal più piccolo al più grande.

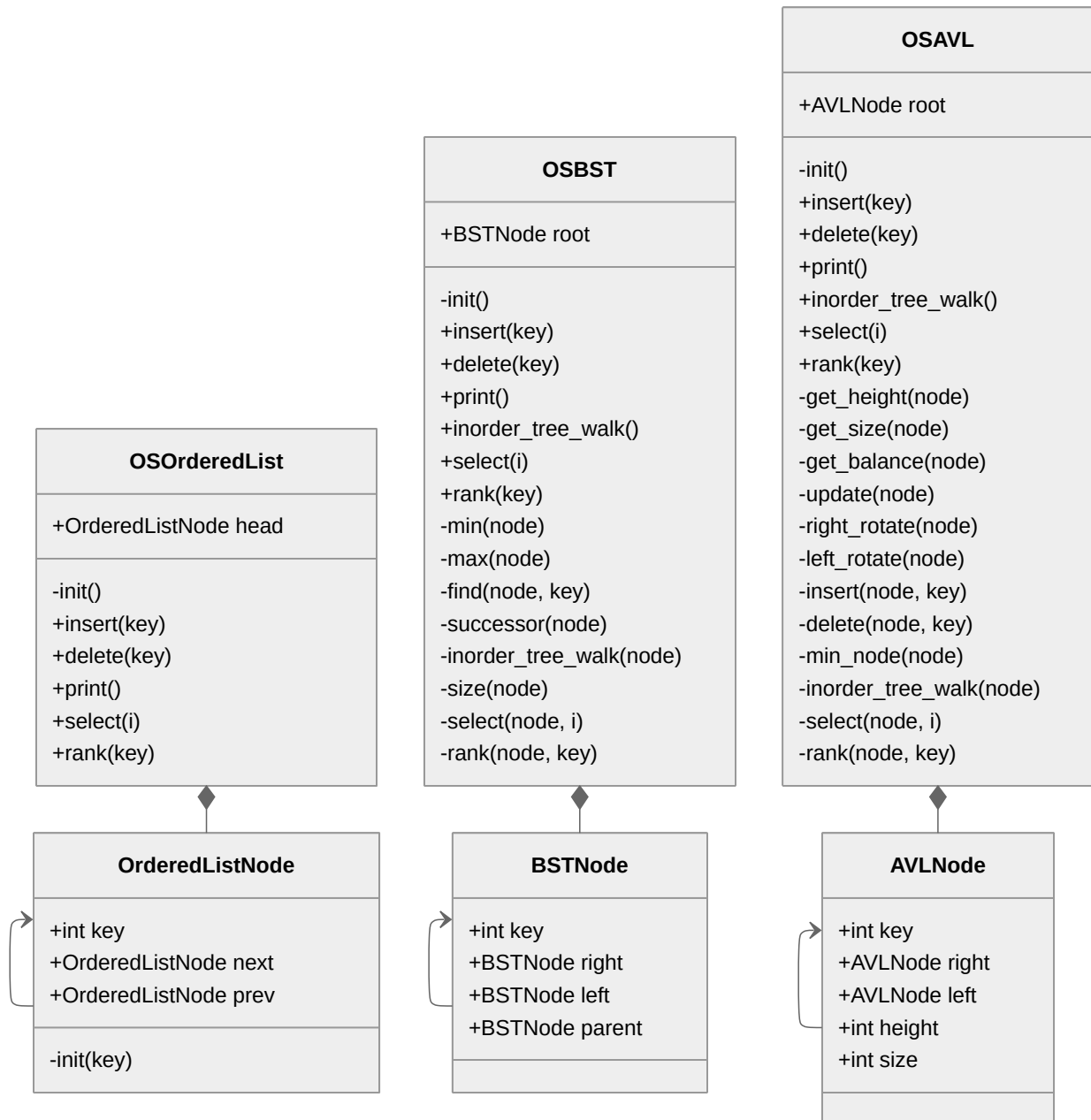


Figura 1: Diagramma UML delle tre implementazioni

3.3 Implementazione degli esperimenti

4 Esperimenti e risultati

4.1 Presentazione dei risultati

4.2 Analisi e discussione

5 Conclusioni